

Bootstrap for Regression and Simulation of Power Curves

Liuxi Mei

Xiaochen Liu

2025-02-20

Question 1: Bootstrap for regression

The dataset `kresseertrag.dat` has three columns: the observation number (1, . . . , 81), the concentration of fertilizer used in %, the yield of cress in mg. The data should be analysed with a regression model, assuming that the errors are independent with equal variance. In the lecture, the straight-line regression was considered; but here, you should use a potentially better model.

Part a

Read in the dataset and fit a cubic regression model

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \epsilon,$$

where x = concentration and y = yield, using the R-function `lm`.

```
data <- read.csv('kresseertrag.dat', header= FALSE)
# Split column names and keep only the first three parts
# Load necessary library
#install.packages('tidyr')
#install.packages('boot')
library(tidyr)
library(boot)

# Use separate() from tidyr to split the column into three parts
data <- separate(data, col = V1, into = c("Col1", "Col2", "concentration", 'yield'), sep = "\\s+")

data[] <- lapply(data, as.numeric)
data <- data[,c(3,4)]
# Fit a cubic regression model
model <- lm(yield ~ concentration+ I(concentration^2)+ I(concentration^3), data=data)

summary(model)
```

```
##
## Call:
## lm(formula = yield ~ concentration + I(concentration^2) + I(concentration^3),
##     data = data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
```

```
## -58.893 -17.142 -3.893 17.716 58.107
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      199.284      5.442  36.616 <2e-16 ***
## concentration      62.634      72.165   0.868  0.3881
## I(concentration^2) -298.766     165.952  -1.800  0.0757 .
## I(concentration^3)  158.664      91.587   1.732  0.0872 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 26.47 on 77 degrees of freedom
## Multiple R-squared:  0.646, Adjusted R-squared:  0.6322
## F-statistic: 46.84 on 3 and 77 DF, p-value: < 2.2e-16
```

Part b

You might improve the model in a. by removing or adding (a) model term(s). For the chosen model, estimate the coefficients in the model together with their 95%-confidence intervals using the R-function `lm`. Create a plot for yield vs. concentration and add the estimated regression curve to the plot. The P values for terms concentration, concentration² and concentration³ are 0.36, 0.07 and 0.08 respectively. The first term can be removed as it poorly affects the result. Thus, the model is updated as:

$$y = \beta_0 + \beta_2 x^2 + \beta_3 x^3 + \epsilon,$$

95% Confidence Interval for the coefficient of I(concentration²) is (-231.8369, -84.86522).

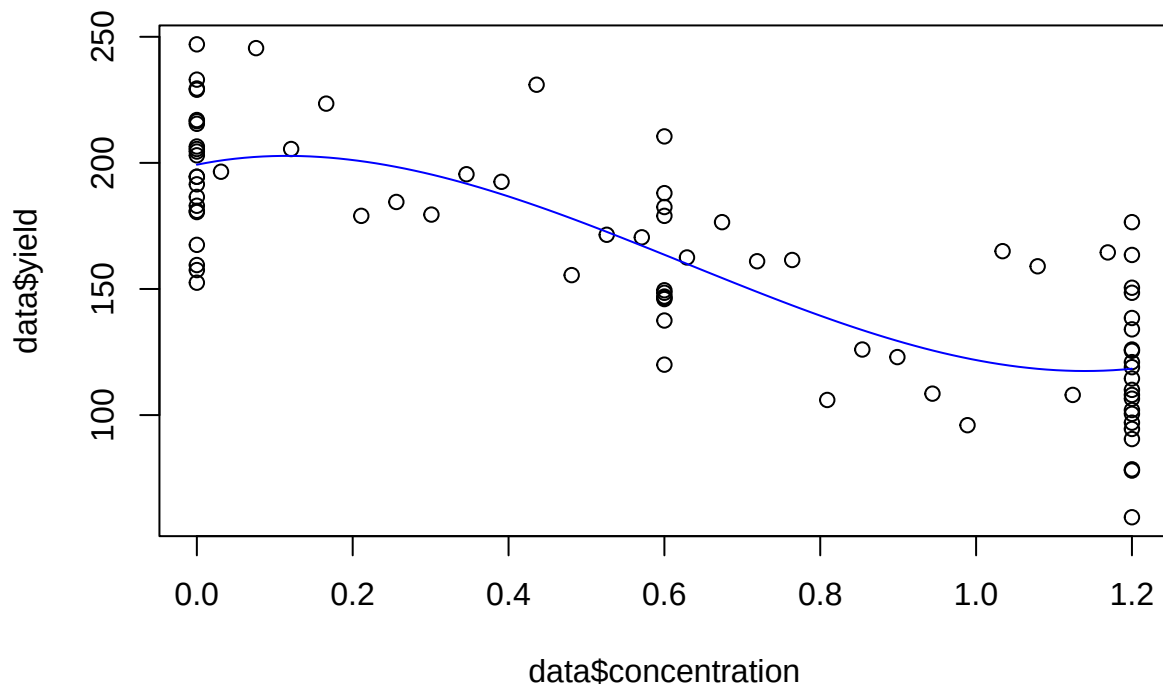
```
model_new <- lm(yield ~ I(concentration^2)+ I(concentration^3), data=data)
coef_intervals<- confint(model_new, level = 0.95)

# Print the 95% confidence interval for the second coefficient
cat("95% Confidence Interval for the coefficient of I(concentration^2):",
    coef_intervals[2, ], "\n")
```

```
## 95% Confidence Interval for the coefficient of I(concentration^2): -231.8369 -84.86522
```

```
plot(data$concentration , data$yield) # Create a plot

# Add the regression curve
x <- seq(min(data$concentration), max(data$concentration), length.out = 100)
y <- predict(model, newdata = data.frame(concentration = x))
lines(x, y, col = "blue")
```

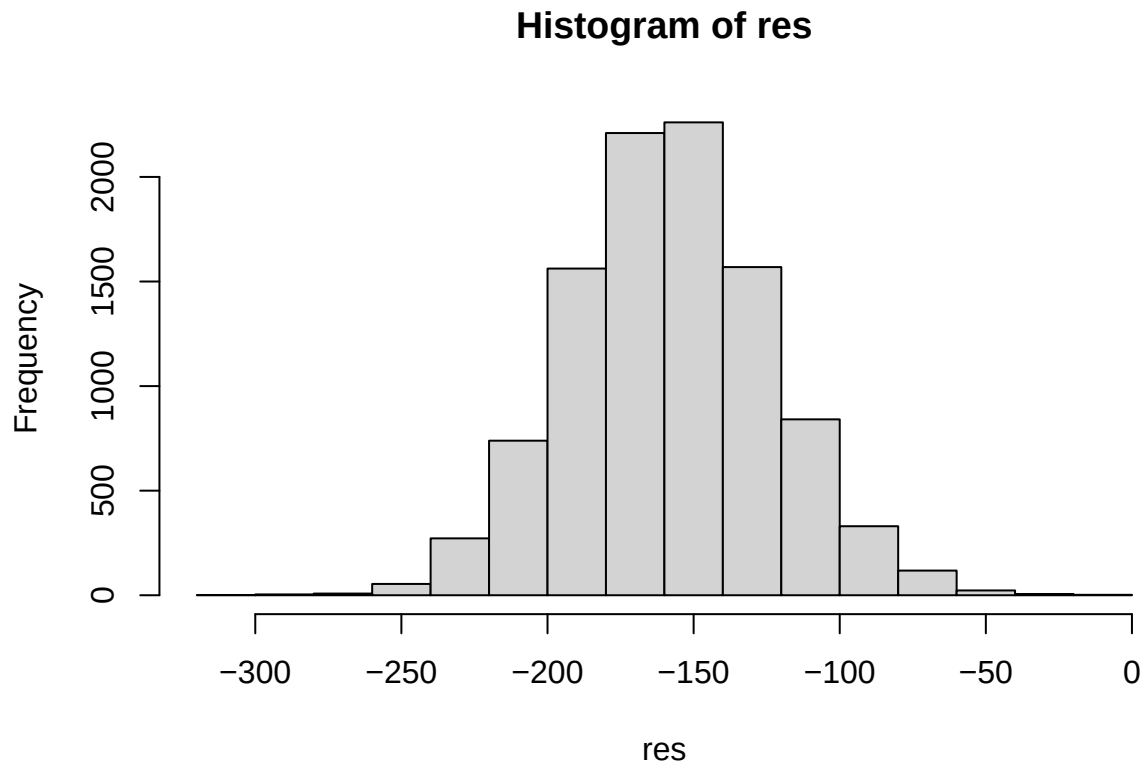


Part c

Derive a 95%-bootstrap confidence interval for one of the model parameters β_i based on the percentile method. Do not use a bootstrap package for this calculation; program the bootstrap on your own. Use at least 10000 bootstrap replicates. Plot a histogram with the bootstrap distribution for the parameter. β_2 , corresponding to the coefficient of concentration^2 is estimated. 95% Confidence Interval for the coefficient of $I(\text{concentration}^2)$ is $(-223.7103, -88.63534)$.

```
res <- c()
bo<- 10000
set.seed(123)
for (i in 1:bo) {
  x <-sample(dim(data)[1],size = dim(data)[1], replace = TRUE)
  cm <- lm(yield~I(concentration^2)+ I(concentration^3), data=data[x,])
  res <- c(res, as.numeric(coef(cm)[2]))
}

hist(res)
```



```
ress <- sort(res)
ci95 <- c(ress[round(bo*0.025)], ress[round(bo*0.975)])
# Print the 95% confidence interval
cat("95% Confidence Interval for the coefficient of I(concentration^2):", ci95, "\n")
```

```
## 95% Confidence Interval for the coefficient of I(concentration^2): -223.7103 -88.63534
```

Part d

Derive now 95%-confidence intervals for the chosen parameter using the package boot with percentile and BCa method. 95% Confidence Interval for the slope is (-226.396, -88.61025).

```
cmslope <- function(data, i){
  cm <- lm(yield~I(concentration^2)+ I(concentration^3), subset=i, data=data)
  coef(cm)[2]
}

set.seed(123)
cb <- boot(data, cmslope, R=10000)
ci <- boot.ci(cb, type="perc")
# Print the result
cat("95% Confidence Interval for the slope:", ci$percent[4:5], "\n")
```

```
## 95% Confidence Interval for the slope: -226.396 -88.61025
```

Part e

Compare the confidence intervals from b., c., and d. and comment on it. Do you judge that the intervals are similar or do you see relevant differences? What is your overall conclusion from these confidence interval results about that model parameter? The results from 3 methods are summarized as follow. The results from manual bootstrap method and from boot package seem to have a good consistence with each other. Those two methods build their CI based on their samples' median values rather than their estimates. The CI result from lm model is based on the theoretical properties of the linear regression model, the normal distribution assumption. As a result, the CI based on Bootstrap methods has a obvious difference from the result from a lm model.

```
# Create a data frame with the results
results <- data.frame(
  Method = c("Standard lm Confidence Interval", "Manual Bootstrap Method",
    "Bootstrap using boot package (Percentile)" ),
  `95% Confidence Interval` = c(
    paste(coef_intervals[2,], collapse = ", "),
    paste(ci95, collapse = ", "),
    paste(ci$percent[4:5], collapse = ", ")
  )
)

# Print the table using knitr::kable
knitr::kable(results, col.names = c("Method", "95% Confidence Interval for
  2 (Coefficient of I(concentration^2))"))
```

Method	95% Confidence Interval for 2 (Coefficient of I(concentration ²))
Standard lm Confidence Interval	-231.836869708589, -84.8652150102764
Manual Bootstrap Method	-223.71034349385, -88.6353435288563
Bootstrap using boot package (Percentile)	-226.395976706178, -88.6102490770488

Question 2: Simulation of power curves

The Gumbel distribution (with scale parameter set to 1 and location parameter $\mu + c$ where $c = \log(\log(2)) \approx -0.36651$) has the distribution function

$$F(x) = \exp(-\exp(-(x - \mu - c))), \quad c = \log(\log(2)).$$

The median of a random variable with this distribution is μ .

Assume that $n = 13$ independent observations are made and they follow a Gumbel distribution with location $\mu + c$. To test the null hypothesis

$$H_0 : \mu = 0 \quad (\text{median is } 0)$$

versus

$$H_a : \mu > 0,$$

one can use the **Sign test**. We want to investigate the power (= probability to reject H_0) of this test in this situation using a simulation study.

Part a

Generate Gumbel random variables using the Inverse Transformation Method.

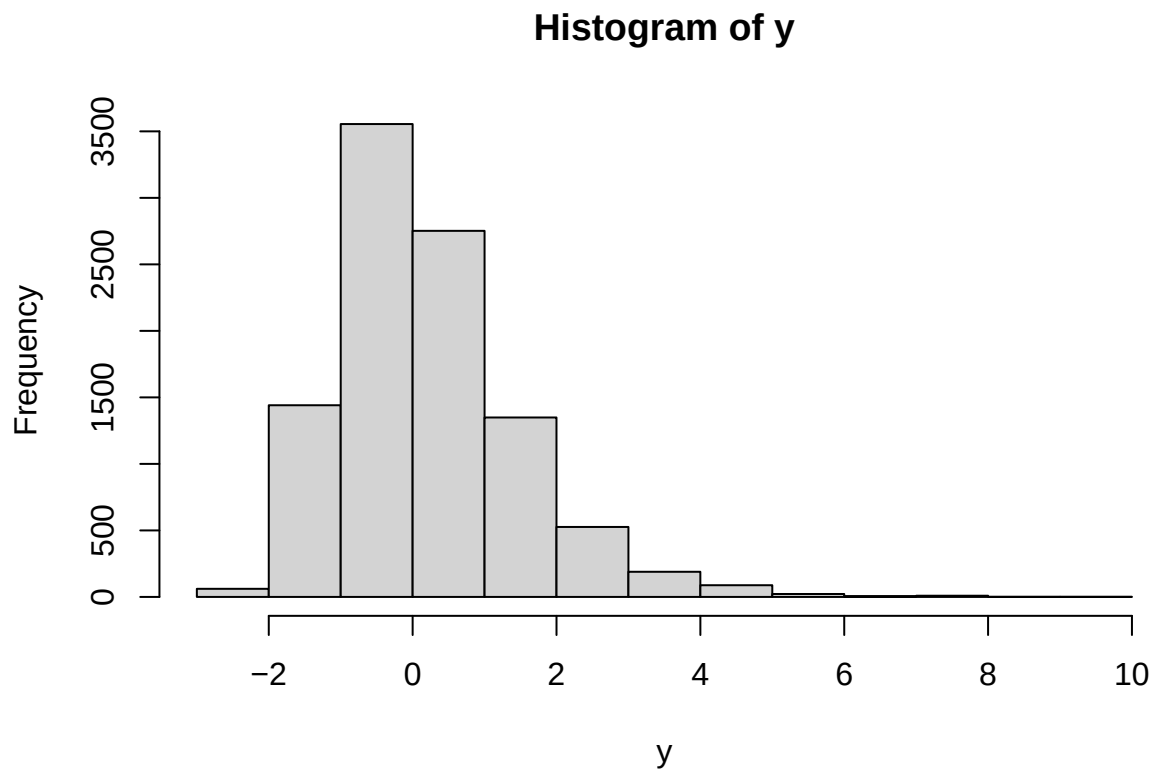
The inverse transformation of the Gumbel distribution with scale parameter set to 1 and location parameter $\mu + c$ (where $c = \log(\log(2)) \approx -0.36651$) is given by:

$$x = -\log(-\log(u)) + \mu + c$$

where: - u is a value between 0 and 1 (quantile), - μ is the location parameter, - $c = \log(\log(2))$.

R Implementation

```
# Define the inverse transformation function for the Gumbel distribution
gumbel_inverse <- function(u, mu) {
  c <- log(log(2))
  x <- -log(-log(u)) + mu + c
  return(x)
}
set.seed(123)
x <- runif(10000)
y <- gumbel_inverse(x, mu=0)
hist(y)
```



Part b

Simulate the power of the Sign test for $\mu \in [0, 2]$ using an appropriate grid for μ -values on $[0, 2]$ and an appropriate number of repetitions for each μ (under consideration of the simulation's precision; provide some reasoning for the chosen number of repetitions).

Plot the power curve (i.e., power versus μ).

For the Sign test, you might use, e.g., the function `SIGN.test` in the package `BSDA`; or alternatively, the function `binom.test` might be used when applied to the number of positive observations.

```
#install.packages('BSDA')
library(BSDA)

##      lattice

##
##      'lattice'

## The following object is masked from 'package:boot':
##
##      melanoma

##
##      'BSDA'

## The following object is masked from 'package:datasets':
##
##      Orange

s <- 10000
n <- 13

mu <- seq(0, 2, length.out = 100)
power_values <- numeric(length(mu))

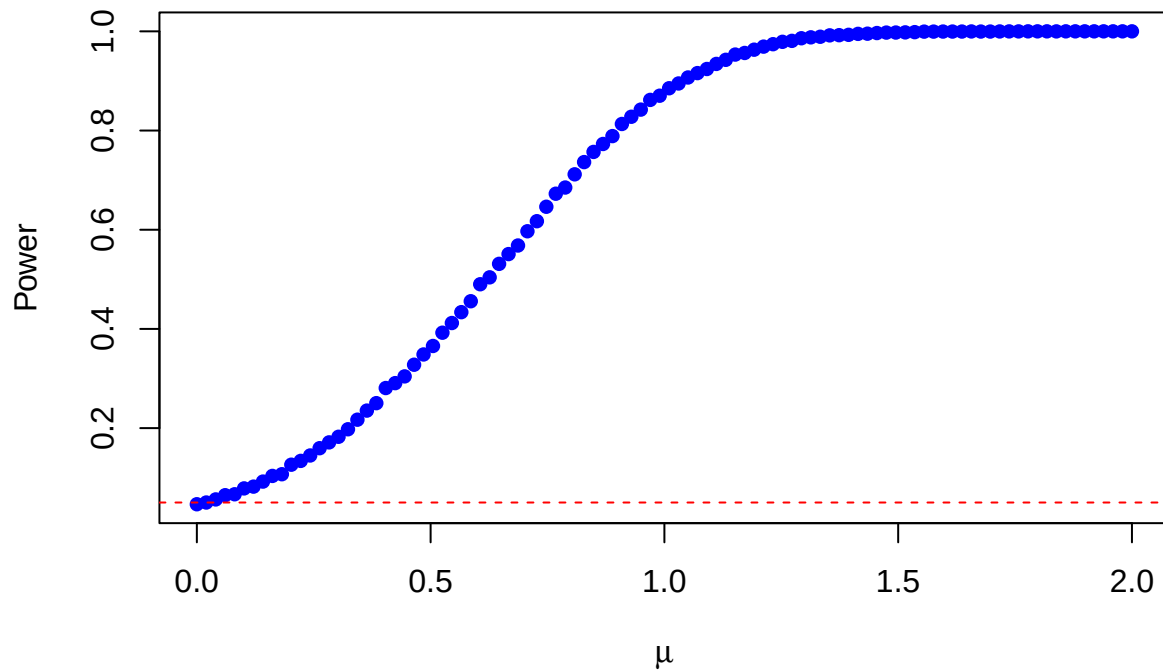
set.seed(123)

for (i in 1:length(mu)) {
  count <- 0
  for (j in 1:s) {

    x <- runif(13)
    y <- gumbel_inverse(x, mu = mu[i])
    p <- SIGN.test(y, alternative = "greater")$p.value
    reject <- (p < 0.05)
    count <- count + reject
  }
  power_values[i] <- count / s
}

plot(mu, power_values, type = "b", pch = 16, col = "blue",
      xlab = expression(mu), ylab = "Power", main = "Power Curve of the Sign Test")
abline(h = 0.05, lty = 2, col = "red") # Reference line at alpha = 0.05
```

Power Curve of the Sign Test



The precision of the estimated power increases with the number of repetitions. Since we have a significant level ($=0.05$), 10,000 repetitions is applied to ensure a good precision and we have stable and reliable estimated power generated, reducing the standard error of the estimate.

Appendix

```
#Assignment 1
data <- read.csv('kresseertrag.dat', header= FALSE)
# Split column names and keep only the first three parts
# Load necessary library
#install.packages('tidyr')
#install.packages('boot')
library(tidyr)
library(boot)

# Use separate() from tidyr to split the column into three parts
data <- separate(data, col = V1, into = c("Col1", "Col2", "concentration", 'yield'), sep = "\\s+")

data[] <- lapply(data, as.numeric)
data <- data[,c(3,4)]
# Fit a cubic regression model
model <- lm(yield ~ concentration+ I(concentration^2)+ I(concentration^3), data=data)

summary(model)
```

```

model_new <- lm(yield ~ I(concentration^2)+ I(concentration^3), data=data)

coef_intervals<- confint(model_new, level = 0.95)

# Print the 95% confidence interval for the second coefficient
cat("95% Confidence Interval for the coefficient of I(concentration^2):",
    coef_intervals[2, ], "\n")

plot(data$concentration , data$yield) # Create a plot

# Add the regression curve
x <- seq(min(data$concentration), max(data$concentration), length.out = 100)
y <- predict(model, newdata = data.frame(concentration = x))
lines(x, y, col = "blue")

res <- c()
bo<- 10000
set.seed(123)
for (i in 1:bo) {
  x <-sample(dim(data)[1],size = dim(data)[1], replace = TRUE)
  cm <- lm(yield~I(concentration^2)+ I(concentration^3), data=data[x,])
  res <- c(res, as.numeric(coef(cm)[2]))
}

hist(res)
ress <- sort(res)
ci95 <- c(ress[round(bo*0.025)], ress[round(bo*0.975)])
# Print the 95% confidence interval
cat("95% Confidence Interval for the coefficient of I(concentration^2):", ci95, "\n")

cmslope <- function(data, i){
  cm <- lm(yield~I(concentration^2)+ I(concentration^3), subset=i, data=data)
  coef(cm)[2]
}

set.seed(123)
cb <- boot(data, cmslope, R=10000)
ci <-boot.ci(cb, type="perc")
# Print the result
cat("95% Confidence Interval for the slope:", ci$percent[4:5], "\n")

# Create a data frame with the results
results <- data.frame(
  Method = c("Standard lm Confidence Interval","Manual Bootstrap Method",
    "Bootstrap using boot package (Percentile)" ),
  `95% Confidence Interval` = c(
    paste(coef_intervals[2,], collapse = ", "),
    paste(ci95, collapse = ", "),
    paste(ci$percent[4:5], collapse = ", ")
  )
)

```

```

# Print the table using knitr::kable
knitr::kable(results, col.names = c("Method", "95% Confidence Interval for
                                     2 (Coefficient of I(concentration^2))"))

#assignment 2
gumbel_inverse <- function(u, mu) {
  c <- log(log(2))
  x <- -log(-log(u)) + mu + c
  return(x)
}
set.seed(123)
x <- runif(10000)
y <- gumbel_inverse(x, mu=0)
hist(y)

install.packages('BSDA')
library(BSDA)
s <- 10000
n <- 13

mu <- seq (0, 2, length.out = 100)
power_values <- numeric(length(mu))

set.seed(123)

for (i in 1:length(mu)) {
  count <- 0
  for (j in 1:s) {

    x <- runif(13)
    y <- gumbel_inverse(x, mu = mu[i])
    p <- SIGN.test(y, alternative = "greater")$p.value
    reject <- (p < 0.05)
    count <- count + reject
  }
  power_values[i] <- count / s
}

plot(mu, power_values, type = "b", pch = 16, col = "blue",
      xlab = expression(mu), ylab = "Power", main = "Power Curve of the Sign Test")
abline(h = 0.05, lty = 2, col = "red") # Reference line at alpha = 0.05

```